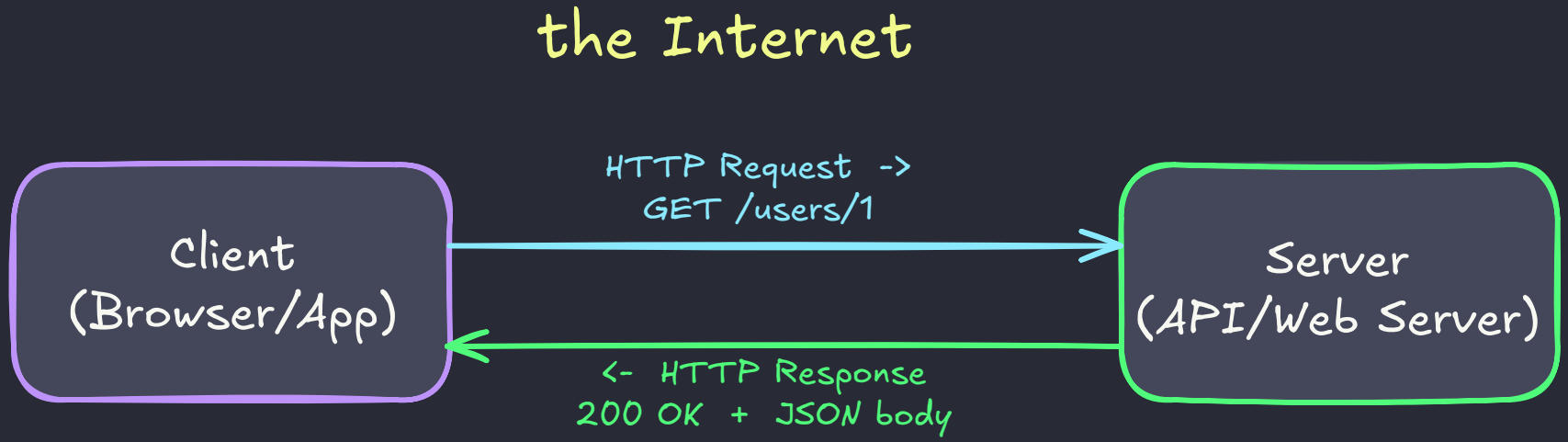
HTTP and REST APIs

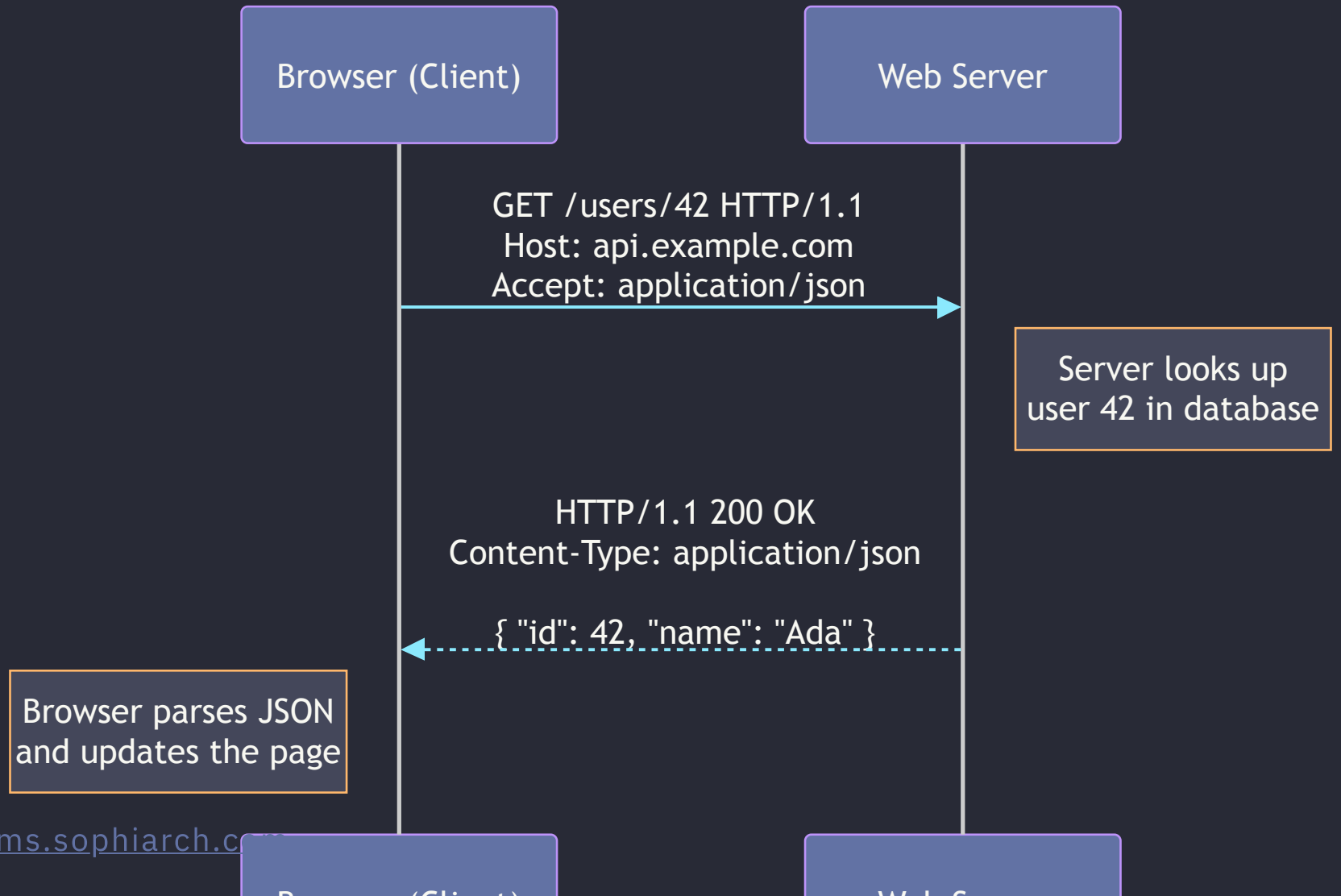
What is HTTP?

- **HyperText Transfer Protocol** — the language browsers and servers speak
- A **client-server** protocol for exchanging resources
  - web pages, images, JSON data, files
- **Stateless**: each request is independent
  - the server remembers nothing between them
- Sits at the **application layer**, carried over **TCP** (transport layer)
  - HTTP defines the message format, TCP guarantees the bytes arrive intact and in order

Client-Server Architecture



The Request-Response Cycle



Anatomy of a URL

`https://api.example.com:8000/users/42?active=true#profile`

`https://`	Scheme — protocol used (http, https)
`api.example.com`	Host — domain/server name
`:8000`	Port — which door on the server (default 443/80)
`/users/42`	Path — resource being addressed
`?active=true`	Query string — filters/params as key=value
`#profile`	Fragment — client-side anchor (not sent to server)

A Raw HTTP Request

```
GET /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
```

- Line 1: **method** + **path** + **protocol version**
- `Host`: which server to talk to (one IP can serve many domains)
- `Accept`: what response format the client wants back

A Raw HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat",
  "body": "quia et suscipit..."
}
```

- Status line: protocol version + **status code** + reason phrase
- Headers describe the body; blank line separates headers from **body**

Common HTTP Methods

Method	Purpose	Has a body?
GET	Retrieve a resource	No
POST	Create a new resource	Yes
PUT	Replace a resource entirely	Yes
PATCH	Partially update a resource	Yes
DELETE	Remove a resource	Usually no

Method Examples — Raw HTTP

```
POST /posts HTTP/1.1
Host: jsonplaceholder.typicode.com
Content-Type: application/json

{ "title": "Hello", "body": "World", "userId": 1 }
```

```
DELETE /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
```

HTTP Status Codes

- **1xx** — Informational (rarely seen directly)
- **2xx** — Success (`200 OK`, `201 Created`, `204 No Content`)
- **3xx** — Redirection (`301 Moved Permanently`)
- **4xx** — Client error (`400 Bad Request`, `401 Unauthorized`, `404 Not Found`)
- **5xx** — Server error (`500 Internal Server Error`)

> *Rule of thumb:* ***4xx = you made a mistake, 5xx = the server did***

Status Code Examples

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 101, "title": "Hello", "body": "World", "userId": 1 }
```

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{ "error": "Resource not found." }
```

JSON — the Language of APIs

- **JavaScript Object Notation** — lightweight, human-readable data format
- Key-value pairs, nested objects and arrays, no code execution
- Almost every modern web API sends and receives JSON

```
{
  "id": 42,
  "name": "Ada Lovelace",
  "skills": ["python", "math"],
  "active": true
}
```

HTTP Headers

- Metadata sent with every request and response
- Tell the server (and client) **how to interpret the data**

Header	Example Value	Purpose
`Content-Type`	`application/json`	Format of the body being sent
`Accept`	`application/json`	Format you want back
`Authorization`	`Bearer sk-abc123`	Identity / API key
`User-Agent`	`python-requests/2.31`	What client is making the call

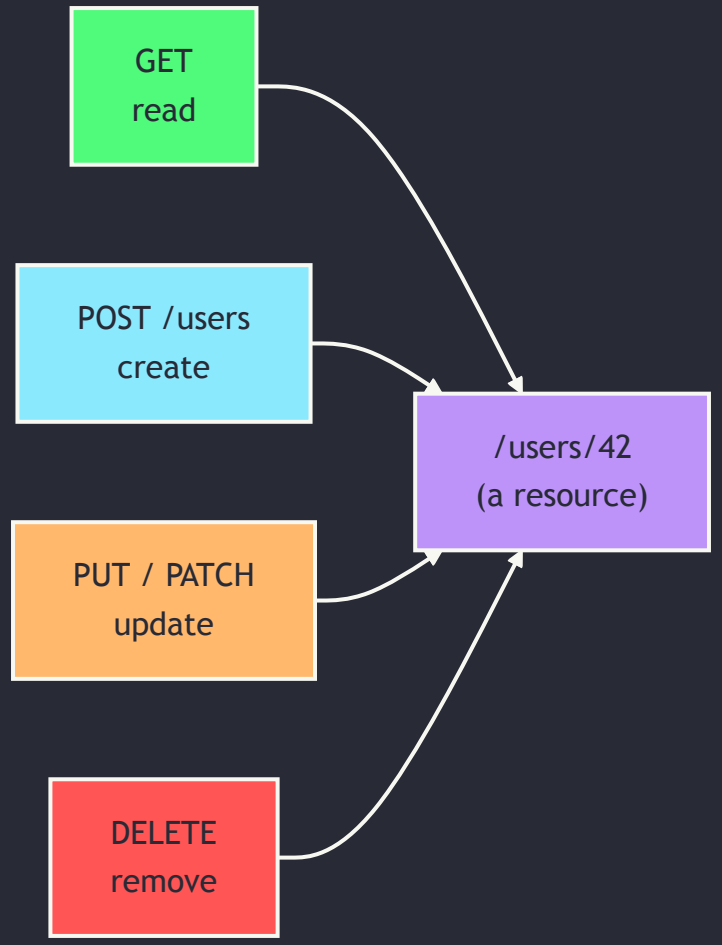
What is an API?

- **Application Programming Interface** — a contract for how software talks to software
- A **Web API** exposes that contract over HTTP
- Client sends a request to an **endpoint** (a URL), server returns structured data
- You don't need to know how the server works inside — only its **interface**

What Makes an API "RESTful"?

- **REST** = **RE**presentational **S**tate **T**ransfer, a design style (not a protocol)
- **Resources** are nouns, identified by URLs — `/users`, `/users/42`, `/users/42/orders`
- **Methods are verbs** — GET/POST/PUT/PATCH/DELETE act on those nouns
- **Stateless** — every request carries everything the server needs (matches HTTP itself)
- Responses typically use **JSON** as the data format

REST Resource, Mapped to HTTP Methods



Testing APIs Without Code — Postman

- **Postman**: a GUI app for building, sending, and inspecting HTTP requests
  - www.postman.com — free desktop app or web version
- No code needed — pick a method, type a URL, hit **Send**
- Great for **exploring** an API before writing a single line of Python
- Also useful for: saving requests in **collections**, sharing with teammates, generating docs

Anatomy of a Postman Request

Sending Requests in Postman

A GET request

1. Click `New` → `HTTP`, select **GET**
2. Enter URL: `https://jsonplaceholder.typicode.com/posts/1`
3. Click **Send** → inspect the response panel

A POST request

1. Change method to **POST**, same base URL without `/1`
2. `Body` tab → `raw` → `JSON`, enter `{ "title": "Hello", "userId": 1 }`
3. Click **Send** → expect `201 Created`

Postman Environment Variables

- Store secrets (API keys, base URLs) as **variables** instead of hardcoding them in requests
- Click **Environments** (top-right) → `+` → add a variable, e.g. `HF_TOKEN` (type: `secret`)
- Reference it anywhere with double curly braces:

```
Authorization: Bearer {{HF_TOKEN}}
```

> *Same idea as `os.environ["API_TOKEN"]` in Python — never paste a real key directly into a request*

Calling an API from Python

```python
import requests

response = requests.get("https://jsonplaceholder.typicode.com/posts/1")

print(response.status_code)   # 200
print(response.headers["Content-Type"])
print(response.json())        # parsed dict, ready to use
```

- `requests` is the standard library for HTTP in Python (`uv add requests`)
- `.json()` parses the body straight into a Python dict/list

Sending Data — POST with a JSON Body

```python
import requests

payload = {"title": "Hello", "body": "World", "userId": 1}
response = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=payload,
)

print(response.status_code)   # 201
print(response.json()["id"])  # new resource id
```

- `json=payload` auto-encodes the dict **and** sets `Content-Type: application/json`
- Compare to `data=payload`, which sends form-encoded data instead — a common source of 400 errors

Query Parameters and Headers in Python

```python
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts",
    params={"userId": 1},                          # -> ?userId=1
    headers={"Authorization": "Bearer YOUR_TOKEN"},
)

for post in response.json():
    print(post["title"])
```

- `params=` builds and URL-encodes the query string for you
- `headers=` sends any custom headers — most often used for authentication

API Authentication Basics

- Most real-world APIs require proof of identity — an **API key** or **token**
- Most common pattern: `Authorization` header with a **Bearer token**

```
Authorization: Bearer YOUR_API_KEY
```

- Common services using this pattern: GitHub, Stripe, OpenAI, HuggingFace
- **Never hardcode keys in source code** — load from environment variables instead

```
import os, requests
headers = {"Authorization": f"Bearer {os.environ['API_TOKEN']}"}
```

Error Handling in Python

```python
response = requests.get("https://jsonplaceholder.typicode.com/posts/9999")

if response.status_code == 200:
    data = response.json()
else:
    print(f"Request failed: {response.status_code} - {response.text}")

# or, raise an exception on any 4xx/5xx:
response.raise_for_status()
```

- Always check `status_code` (or use `raise_for_status()`) before trusting `.json()`
- `.text` shows the raw response body — invaluable for debugging error messages

HTTP + REST in Practice

You will use this to:

- Fetch data from public APIs
- Submit forms and data to a backend
- Call ML/LLM inference endpoints
- Build your own backend routes later

Typical flow, every time:

1. Pick the right method + URL
2. Add headers (auth, content-type)
3. Send params or a JSON body
4. Check `status_code`
5. Parse `.json()` and use the data

Common Pitfalls

- **Forgetting** `Content-Type` when sending a JSON body — server can't parse it correctly
- **Using** `data=` **instead of** `json=` in `requests` — sends the wrong encoding
- **Not checking** `status_code` before calling `.json()` — crashes on error responses
- **Hardcoding API keys** in scripts — security risk, breaks if committed to git
- **Confusing PUT and PATCH** — PUT replaces the whole resource, PATCH updates part of it

Cheat Sheet

Action	Python (`requests`)
GET request	`requests.get(url)`
GET with query params	`requests.get(url, params={...})`
POST with JSON body	`requests.post(url, json={...})`
Add headers / auth	`requests.get(url, headers={...})`
Check success	`response.status_code`, `response.raise_for_status()`
Parse JSON response	`response.json()`
Raw response text	`response.text`

References

Topic	Source
HTTP/1.1 specification	Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. RFC 7231, IETF.
REST architectural style	Fielding, R. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, Chapter 5). University of California, Irvine.
MDN HTTP documentation	Mozilla. (2026). HTTP — MDN Web Docs.
`requests` **library docs**	Reitz, K. (2026). Requests: HTTP for Humans.
HTTP status codes reference	Mozilla. (2026). HTTP response status codes — MDN Web Docs.
Test API for practice	typicode. (2026). JSONPlaceholder — Free fake API for testing.
Postman documentation	Postman, Inc. (2026). Postman Learning Center.

Recap & Practice

- HTTP is a stateless client-server protocol: request out, response back, every time
- REST names resources as URLs
  - uses HTTP methods as the verbs that act on them
- `requests.get/post(url, params=, json=, headers=)`
  - covers nearly every API call you'll write